

🔒 講師用 — 受講者には配布・公開しないでください(解答例を含みます)

FDE研修 新卒コース 講師用ガイド・演習解答例

進行ノート/口頭試問の想定問答/全演習の模範解答

対応教材:docs/fde-training-newgrad-handbook.html(受講者用・全9章)/ 想定講師:メンターFDE + 顧客役マネージャー

講師の役割と運営原則

- 新卒コースの受講者は「書ける」自信を持って入ってきます。講師の仕事は**自信を折らずに「実務の基準」**を見せること。差し戻しには必ず「実務でこうなる理由」を添えます。
- **口頭試問を多用する**。本コースの合格基準の多くは「説明できること」。各章の想定問答を本ガイドに収録しています。
- 優秀な受講者ほど個人プレーに走ります。第9章の模擬プロジェクトでは「**チームの成果=個人の評価**」を初日に明言してください。

第1章 チーム開発の作法 — 講師ノート

毎日1件のIssue→PRサイクル。講師は「レビューの品質」と「Issueの粒度」を管理します。

Issue・PRの添削基準

📌 差し戻し判断の目線

受講者がやりがちなこと	差し戻しコメント例
完了条件のないIssue(「～を改善する」だけ)	「このIssue、何ができたら閉じられますか?チェックボックスで書いてみましょう」
Issueより先にコードを書いてしまう	「実務では着手前のIssueが工数見積りと合意の根拠になります。今回は事後でよいので起票を」
1つのPRに複数Issueの変更を混ぜる	「レビュアーは差分の理由を1つずつ確認します。#23と#25は別PRに分割を」
PRに動作確認方法がない	「レビュアーがこのPRを検証する手順を書いてください。書けない場合、検証していない可能性が高いです」

💡 相互レビュー演習では、講師が意図的に「軽微な問題のあるPR」を1本投稿し、受講者にレビューさせるのが効果的です(仕込み例:マジックナンバー、コメントと実装の不一致、encoding未指定)。

第2章 コードリーディングー 想定問答集

発表会での口頭試問が本章の実質的な試験です。以下の質問から3問以上を出題してください。

口頭試問:質問と期待回答

📌 想定問答(教材アプリの実コードに基づく)

質問	期待回答(要旨)
「回答の文体を変えたい。どのファイルに触る？」	constants.py(システムプロンプトが定数として集約されている)。components.py と迷ったら「表示」と「生成」の違いを確認
「検索対象のフォルダを増やしたい場合は？」	constants.py のデータフォルダ設定と、initialize.py の読み込み処理。両者の関係を説明できれば合格
「main.py はなぜこんなに短いのか？」	流れの制御だけを担い、処理の本体は utils / components / initialize に委譲しているから(責務分離)。「main.py に全部書いたら何が困るか」まで言えたら加点
「st.session_state を使わないとどうなる？」	Streamlitは操作のたびにスクリプトを再実行するため、会話履歴やretrieverが毎回消えて作り直しになる(重い初期化が毎回走る)
「2つの利用モードは内部で何が違う？」	utils.get_llm_response() 内でモードによりシステムプロンプトと検索件数(k)を切り替えている

第3章 テスト・ログ — 解答例と発展指導

教材掲載の4テストは写経で書けます。講師の仕事は「その先」の議論に引き上げることです。

追加テストの模範解答(受講者が自分で増やす分)

模範解答:追加しておきたい境界ケース

```
# 追加1: https でもリンク扱いになるか(startswith("http") は https も含む)
def test_get_source_icon_https():
    assert utils.get_source_icon("https://portal.example.co.jp") == ct.LINK_SOURCE_

# 追加2: 空文字はファイル扱いに倒れる(現仕様の確認テスト)
def test_get_source_icon_empty():
    assert utils.get_source_icon("") == ct.DOC_SOURCE_ICON

# 追加3: build_error_message は複数行メッセージでも連結順序を保つか
def test_build_error_message_multiline():
    result = utils.build_error_message("1行目\n2行目")
    assert result.index("2行目") < result.index(ct.COMMON_ERROR_MESSAGE)
```

議論ポイント:追加2は「バグの検出」ではなく「現仕様の文書化」であること。「空文字が来たら本当はどうすべきか?」という仕様の議論にテストから入る体験をさせてください。

発展の指導:get_llm_response() はなぜ難しいか

期待する説明と、興味を持った受講者への提示

- 期待回答:「LLM API(外部・非決定的・課金)と st.session_state(画面状態)に依存しており、入力に対して出力が一意に決まらないから」。
- 発展提示:依存を「差し替え可能」にすれば一部はテストできる(モックの概念)。深追いは研修範囲外とし、「テストしにくい関数は設計を見直すサイン」という原則だけ持ち帰らせます。

第4章 RAGチューニング評価 — 採点基準

最頻出の失敗は「2つのパラメータを同時に変える」。実験計画の段階(火曜まで)で講師が計画表をチェックしてください。

評価レポートの採点基準

📌 採点ループリック

観点	合格(2点)	優秀(3点)
実験設計	1実験1変数を守り、基準条件との比較になっている	質問セットに「わざと答えられない質問」を混ぜ、拒否できるかも評価している
採点の客観性	正解つき質問セットで、採点基準が事前に定義されている	自分以外の人が採点しても同じ点になる粒度で基準が書かれている
結論	推奨設定と根拠が数字で示されている	「データの性質(表形式が多い等)によって最適値が変わる」という一般化に到達している
コスト意識	—	Embeddingモデル変更のコスト影響(再インデックス費用・応答時間)に言及している

講評で必ず聞く質問:「顧客に『とにかく精度を上げて』と言われたら、次に何を試す?」— 期待回答はパラメータではなく「**まずデータの品質と質問の傾向を見る**」。パラメータいじりに閉じない視野を確認します。

第5章 オントロジー — 設計演習の模範解答

SDK検証スクリプトは foundry-platform-sdk のインストールさえ通れば全PASSします。FAILが出る場合はSDKバージョンを確認(検証済みバージョンを研修前に固定しておくこと)。

模範解答:data/ フォルダのオントロジー設計

🔍 設計例(受講者の解がこれと異なっても、Link の向きと Action の妥当性が説明できれば正解)

要素	模範解答
Object(5つ)	社員(Employee)/部署(Department)/顧客(Customer)/サービス(Service)/会議(Meeting)
Link	社員 — 所属 → 部署/顧客 — 契約 → サービス/社員 — 出席 → 会議/会議 — 議題対象 → 顧客・サービス(議事録から抽出)
Action	社員を異動させる(所属Linkの付け替え)/契約を更新・解約する/会議の決定事項を承認する
プロパティ例	社員:社員ID(主キー)・氏名・役職・入社年/会議:開催日・会議名(第6章のパイプラインで抽出する項目と一致させると美しい)

比較考察の期待回答:「RAGだけでは解けない質問」

🔍 期待する例

- 「営業部の社員が今月出席した会議の数は?」 — 集計・横断はRAG(文書断片の検索)が苦手。オントロジーなら Link をたどって数えるだけ
- 「A社の契約と、A社が議題に出た会議を時系列で並べて」 — 複数Objectの関係の追跡
- 逆に「出張旅費の規程の詳細は?」のような文書の中身を読む質問はRAGが得意。「両者は競合ではなく補完」に到達させるのが本章のゴールです。

第6章 データパイプライン — 抽出関数の解答例

教材は骨格のみ提示しています。抽出関数の模範実装は以下のとおり(受講者の正規表現がこれより素朴でも、テストデータで動けば合格)。

模範解答:extract_date / extract_title / extract_members / validate

```
import re

DATE_PATTERNS = [
    (re.compile(r"(\d{4})年(\d{1,2})月(\d{1,2})日"), "{0}-{1:0>2}-{2:0>2}"),
    (re.compile(r"(\d{4})[/-](\d{1,2})[/-](\d{1,2})"), "{0}-{1:0>2}-{2:0>2}"),
]

def extract_date(text):
    # 表記ゆれ(2026年6月3日 / 2026/06/03 / 2026-6-3)を吸収してISO形式に正規化
    for pattern, fmt in DATE_PATTERNS:
        m = pattern.search(text)
        if m:
            return fmt.format(*m.groups())
    return None # 見つからない場合は None(validateで検知)

def extract_title(text):
    # 先頭の空でない行を会議名とみなす(議事録の様式に依存する割り切りを明記させる)
    for line in text.splitlines():
        if line.strip():
            return line.strip()
    return None

def extract_members(text):
    # 「出席者: 山田、佐藤 鈴木」等から人名リストを抽出(区切りゆれを吸収)
    m = re.search(r"出席者[:,]\s*(.+)", text)
    if not m:
        return []
    return [name.strip() for name in re.split(r"[、,/\.\s]+", m.group(1)) if name.s

def validate(record):
    # 必須項目の欠損は例外にして、呼び出し側の errors リストに集約させる
    missing = [k for k in ("開催日", "会議名") if not record[k]]
    if missing:
        raise ValueError(f"必須項目が抽出できません: {missing}")
```

講評の要点:①「様式に依存する割り切り」(extract_title)をコメントや品質メモで自覚的に書けているか ②名寄せ(山田/山田太郎)をどこまで自動化したかの判断根拠 ③失敗ファイルを「手動確認リスト」として出力する設計になっているか。100%自動化を目指して溶けている受講者には「機械9割+人手1割」の判断を促してください。

第7章 クラウドデプロイ — 審査チェックリスト

デプロイ自体は手順どおりで完了します。講師は「公開してよい状態か」の審査役です。

🔍 講師審査チェックリスト(デプロイ許可の条件)

- ☐ リポジトリの全履歴にAPIキー・.envが含まれない(`git log --all --oneline -- .env` と Secret Scanning の結果を確認)
- ☐ `data/` が研修用サブセットである(実データ・個人情報を含むデータをクラウドに上げていない)
- ☐ Secretsがコードでなく管理画面に設定されている
- ☐ 公開URLの共有範囲を研修内に限定し、研修終了時の停止担当が決まっている
- ☐ 更新→自動再デプロイの確認までやりきった(「デプロイは1回イベント」でなく「継続する仕組み」の理解)

よくある事故と対処:requirements の記載漏れ(ビルドログの `ModuleNotFoundError` を自力で読ませる)/メモリ超過(データ削減で対処。「無料枠の制約も顧客説明の材料になる」と接続する)。

第8章 コンサル実践 — 顧客役の演技ガイド

3回面談ロールプレイの顧客役ガイドです。演技の一貫性が採点の公平性に直結するため、顧客役は全受講者に対して同じカードを使ってください。

📄 顧客役カード(受講者には非公開)

面談	演技指示	採点ポイント
1回目	「資料探しが大変」とだけ語る。具体的な質問には答えるが、数字は「えーっと…」と考えてから出す(即答しない)	事実(頻度・時間・場所)を数字で引き出したか/その場で解決策を語り始めたら減点
2回目	開始10分後に必ず投下:「上司がスマホ対応も必須だと。あと納期、来週にならない?」— 受講者が説明しても一度は食い下がる	即答での安請け合い(減点)/影響の説明+優先順位の質問返し(合格)/代替案・段階案の提示(加点)
3回目	再提案が合理的なら納得する。合意事項の読み合わせを受講者から促されなければ、こちらからも言わない(=促し忘れを記録)	合意の文書化と読み合わせを自分から実施したか

録画振り返りでは「2回目の面談で最初に発した一言」を全員で見ます。「無理です」「できます」のどちらが出たかが最大の教材になります。

シャドーイング受け入れ側への依頼(講師から先輩FDEへ)

- 同席前に「今日の見どころ」を30秒で伝える(例:「今日は宿題の引き取り交渉があるはず」)
- 会議後5分だけ、受講者の気づきメモの「解釈」に対して答え合わせをする
- 顧客に対しては「研修中のメンバーが同席します」と事前に一言(無断同席はNG)

第9章 模擬プロジェクト — 運営ガイド

講師の立場は「顧客役(マネージャー)」と「メンター」の2役を別人で。メンターは口を出しすぎないこと——介入基準を事前に決めておきます。

チーム編成と介入基準

📌 運営ルール

- **編成:**第1～7章の評価をもとに、技術上位者が1チームに固まらないよう配置。仲良しペアは意図的に分ける
- **介入する:**①タスクボードが2日間更新されない ②特定メンバーがPRゼロのまま3日経過 ③顧客役へのデモが「機能紹介」になっている
- **介入しない:**設計の選択ミス(小さく失敗させて中間デモで気づかせる)/チーム内の健全な衝突
- **顧客役の演技:**週次デモでは毎回1つ本気のダメ出し+1つ本気の称賛。3週目は「上司を連れてきた」設定で初見の審査員を追加すると本番感が出ます

最終発表会の採点表(審査員配布用)

項目(各5点)	満点の状態
課題設定と要件定義	実在課題の当事者が「これは自分の困りごとだ」と認める設定になっている
デモ	業務シナリオの物語でデモし、途中の失敗にも落ち着いて対処した
実装品質	PR履歴から全員の貢献が確認でき、レビューの往復が機能している
質疑応答	全メンバーが最低1回発言し、分からない質問を誠実に持ち帰った
振り返り	「次のプロジェクトで変えること」が個人・チーム両方の粒度で具体的

📌 **修了判定:**未経験者コースと同じく3軸ルーブリックで判定会議を実施。模擬プロジェクトは「チーム評価」と「個人評価」を分離し、個人評価はPR履歴・発表での役割・360度フィードバックから行います。